Introduction

Every Python developer eventually writes `some_tuple.append(4)` out of habit and gets slapped with an `AttributeError`. It looks like a typo. It isn't. It's Python enforcing a rule that goes to the core of how the language represents data in memory, and it's one of the most common conceptual gaps interviewers probe for — not because the syntax is hard, but because it separates people who memorized "lists are mutable, tuples aren't" from people who actually understand *why*.

This distinction matters far beyond trivia. It explains why some objects can be dictionary keys and others can't, why passing a list into a function can silently modify the caller's data while passing a tuple can't, and why a function default argument bug (`def f(x=[])`) is a rite of passage for every Python developer. Once you understand mutability at the object level, a whole category of "mysterious" bugs stops being mysterious.

Problem Overview

Here's the puzzle in its simplest form: you have two collections that look almost identical on the surface, a list and a tuple. You try the same operation, adding an element after creation, on both.

```
my_list = [1, 2, 3]
my_list.append(4)   # works fine

my_tuple = (1, 2, 3)
my_tuple.append(4)  # AttributeError: 'tuple' object has no attribute 'append'
```

The question isn't "how do I fix the error" — you can't append to a tuple, full stop. The real question is: what property of a list makes this possible, and what property of a tuple makes it impossible? Answering that requires understanding what a Python variable actually is, and what happens in memory when you "change" an object.

Example

Let's make the invisible visible using `id()`, which returns the memory address of an object.

```
my_list = [1, 2, 3]
print(id(my_list))      # e.g. 140712834558912
my_list.append(4)
```

```
print(id(my_list))      # 140712834558912 — identical
print(my_list)           # [1, 2, 3, 4]
```

The id never changes. Python didn't create a new list and reassign `my_list` to point at it — it reached into the existing object at that memory address and rewrote its internal contents.

Now compare with a tuple:

```
my_tuple = (1, 2, 3)
print(id(my_tuple))     # e.g. 140712834561600
my_tuple = my_tuple + (4,)
print(id(my_tuple))     # different address entirely
print(my_tuple)          # (1, 2, 3, 4)
```

Since a tuple has no in-place way to grow, "adding" an element actually means building a brand-new tuple and pointing the variable at it. The old tuple `(1, 2, 3)` still exists unchanged somewhere in memory until garbage collected.

Intuition

Here's the mental model that makes everything else click: **a variable is not a box that holds a value — it's a label stuck onto an object that lives somewhere else in memory.**

When you write `my_list.append(4)`, you're not touching the label at all. You're following the label to the object it points at and modifying that object directly. The label (`my_list`) never moves.

An object is *mutable* if it exposes some way to modify its internal state without creating a new object — methods like `.append()`, `.pop()`, `.sort()`, or index assignment (`my_list[0] = 99`) all edit the existing object in place.

An object is *immutable* if no such mechanism exists. Once a tuple, string, int, or frozenset is built, there's no operation Python offers to reach in and rewrite its slots. Any "modification" necessarily means constructing an entirely new object.

This isn't an accident of API design — it's a deliberate contract the object type makes with everyone holding a reference to it. If you hand a list to another function, that function can mutate it and you'll see the change, because you're both pointing at the same box. If you hand a tuple to another function, it *cannot* be mutated out from under you, because no operation exists that would let it.

Brute Force Solution

There isn't really a "brute force vs optimal" axis for a conceptual question like this, but if you want a naive way to *demonstrate* mutability without knowing about `id()`, you could compare objects for equality before and after an operation and assume unchanged identity:

Idea: Print the object, mutate it, print again, and eyeball whether the reference "still feels like" the same object.

Algorithm: 1. Print the collection. 2. Attempt the modification. 3. Print the collection again. 4. Infer mutability from whether the operation succeeded.

Advantages: Requires no special functions, easy to explain to a beginner.

Disadvantages: Doesn't actually prove anything about memory — it only observes that values changed, not whether the underlying object is the same. You could be fooled by reassignment, which also changes printed output without mutating anything.

Time/space complexity: O(1) for the check itself; not really the point of this exercise since it's conceptual rather than algorithmic.

Optimal Solution

The rigorous way to prove mutability is exactly what the video demonstrates: compare `id()` before and after the operation.

Step	List	Tuple
Create object	`id` = A	`id` = B
Attempt modification	`.append()` succeeds	`.append()` doesn't exist
Check `id()` after	Still A	N/A — operation fails, or if you concatenate, new tuple gets `id` = C
Conclusion	Same box, new contents → **mutable**	New box required for any change → **immutable**

The `id()` function is the ground truth here because it reports the actual memory address the interpreter allocated for the object — nothing about print statements or `==` comparisons can fake this out. Two lists with the same contents but different identities will have different `id()` values and `is` will return `False` between them, even though `==` returns `True`.

This also explains **hashability**. Python's dict and set implementations compute a hash of a key when it's inserted, and use that hash to file the object into a bucket for fast lookup. That hash is computed once and must remain valid for the object's entire lifetime in the collection — if the object's contents

could change after insertion, its hash would go stale and the dict would silently become unable to find its own keys.

```
d = {(1, 2): "point A"}      # works — tuple is immutable, hash is stable
d = {[1, 2]: "point A"}      # TypeError: unhashable type: 'list'
```

Tuples (and strings, ints, frozensets) implement `__hash__` because they can guarantee their contents never change. Lists deliberately do not implement `__hash__` — Python protects you from creating dictionary keys that could quietly become inconsistent.

Python Code

```
def demonstrate_mutability():
    """Show identity-preserving mutation on a list vs. forced
    reallocation on a tuple."""
    my_list = [1, 2, 3]
    list_id_before = id(my_list)
    my_list.append(4)
    list_id_after = id(my_list)

    my_tuple = (1, 2, 3)
    tuple_id_before = id(my_tuple)
    try:
        my_tuple.append(4)  # will raise
    except AttributeError:
        pass
    tuple_id_after_failed_mutation = id(my_tuple)

    # The only way to "grow" a tuple is to build a new one
    new_tuple = my_tuple + (4,)
    tuple_id_after_rebuild = id(new_tuple)

    return {
        "list_same_object": list_id_before == list_id_after,
        "tuple_unchanged_after_failed_append": (
            tuple_id_before == tuple_id_after_failed_mutation
        ),
        "rebuilt_tuple_is_new_object": tuple_id_before != tuple_id_after_rebuild,
    }


def hashable_check(value):
    """Return True if value can be used as a dict key / set member."""
    try:
        hash(value)
        return True
    except TypeError:
        return False
```

```
if __name__ == "__main__":
    result = demonstrate_mutability()
    assert result["list_same_object"] is True
    assert result["tuple_unchanged_after_failed_append"] is True
    assert result["rebuilt_tuple_is_new_object"] is True

    assert hashable_check((1, 2)) is True
    assert hashable_check([1, 2]) is False
    assert hashable_check("point") is True

    print("All mutability and hashability checks passed.")
```

Code Walkthrough

- `list_id_before = id(my_list)` captures the memory address before any mutation, our baseline.
- `my_list.append(4)` mutates the list in place — no new object is created.
- `list_id_after == list_id_before` confirms the object identity never changed, proving the append happened in-place.
- `my_tuple.append(4)` is wrapped in a `try/except` because tuples genuinely have no `append` method — this isn't a runtime restriction, it's the absence of a method entirely.
- `new_tuple = my_tuple + (4,)` is the *only* way to get "an extended tuple" — concatenation always builds a new tuple object.
- `tuple_id_before != tuple_id_after_rebuild` proves that "growing" a tuple necessarily means a new memory address, not an in-place edit.
- `hashable_check()` calls `hash()` inside a try block — this is the actual mechanism Python uses internally when you use a value as a dict key or add it to a set. If `__hash__` isn't implemented (as with `list`), it raises `TypeError` immediately.

Dry Run

Trace `demonstrate_mutability()` step by step:

1. `my_list = [1, 2, 3]` → allocated at address, say, `0x100`.
2. `list_id_before = 0x100`.
3. `my_list.append(4)` → list becomes `[1, 2, 3, 4]`, still at `0x100`.
4. `list_id_after = 0x100` → matches `list_id_before`.
5. `my_tuple = (1, 2, 3)` → allocated at, say, `0x200`.

6. `tuple_id_before = 0x200`.
7. `my_tuple.append(4)` → raises `AttributeError`, caught, nothing changes.
8. `tuple_id_after_failed_mutation = 0x200` → unchanged, as expected since nothing succeeded.
9. `new_tuple = my_tuple + (4,)` → Python builds a fresh tuple `(1, 2, 3, 4)` at a new address, say `0x300`.
10. `tuple_id_after_rebuild = 0x300` → different from `0x200`, confirming a new object was necessary.

All three assertions pass, matching the theory exactly.

Complexity Analysis

- `id()` lookups: O(1) — it's just reading an attribute of the object header.
- `list.append()`: amortized O(1) — Python over-allocates list capacity so most appends don't require a reallocation.
- Tuple concatenation (`tuple + tuple`): O(n) — a new tuple of combined length must be allocated and both source tuples copied into it.
- `hash()` on a tuple: O(n) in the size of the tuple (it must hash each element and combine), but for interview purposes this is treated as effectively O(1) for small fixed-size tuples like coordinates.
- Space: list append is O(1) amortized extra space; tuple "growth" is always O(n) extra space since the whole new tuple must be materialized.

These complexities aren't incidental — they're a direct consequence of the mutability model. Mutable objects can amortize costs across in-place edits; immutable objects pay full reconstruction cost on every "change" because reconstruction is the only operation available.

Alternative Solutions

If you need tuple-like immutability but with named fields and better readability, `collections.namedtuple` or a `frozen=True` dataclass are the idiomatic alternatives:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Point:
    x: int
    y: int
```

This is still immutable (attempting `point.x = 5` raises `FrozenInstanceError`), still hashable by default, but gives you named access instead of positional indexing. It's strictly better than a raw tuple when the fields have meaning beyond position — use a plain tuple when the shape truly is anonymous, like a lightweight coordinate pair passed briefly between two functions.

Edge Cases

- **Empty tuple/list**: `()` and `[]` behave identically under this model — the emptiness doesn't change the mutability rules at all.
- **Single-element tuple**: `(4,)` — easy to forget the trailing comma; `(4)` is just an int in parentheses, not a tuple.
- **Nested mutable object inside a tuple**: `t = ([1, 2], 3)` — the tuple's *slots* are locked (you can't do `t[0] = something_else`), but the list living in slot 0 can still be mutated: `t[0].append(99)` works fine and `id(t)` never changes. This is the shallow-immutability gotcha — immutability protects the container's structure, not necessarily the mutable objects it references.
- **Attempting to hash a tuple containing an unhashable element**: `hash(([1,2], 3))` still raises `TypeError`, because Python has to hash every element, and the nested list has no `__hash__`.
- **Large lists vs. large tuples**: appending to a huge list stays cheap due to amortized growth; "appending" to a huge tuple via concatenation gets expensive fast since the entire tuple is copied every time.

Common Mistakes

1. **Assuming a tuple makes everything inside it permanently frozen.** It only locks its own slots — nested lists inside a tuple remain fully mutable.
2. **Using a mutable default argument**, e.g. `def f(x=[])`. Since the default is created once at function definition time, every call that doesn't pass `x` shares and mutates the same list.
3. **Confusing `==` with `is`.** Two lists with identical contents but different identities are `==` but not `is` — conflating them leads to subtle bugs when identity actually matters (e.g. checking if two variables reference the same object).
4. **Trying to use a list as a dictionary key** and being surprised by `TypeError: unhashable type`, instead of reaching for a tuple.
5. **Believing `id()` changes mean the values changed, or vice versa.** They're orthogonal — reassigning a variable to an equal-but-different object changes `id()` without necessarily

changing the printed value, and in-place mutation changes the printed value without changing `id()`.

6. **Forgetting the single-element tuple comma**, silently creating an int instead of a tuple, then getting confusing errors much later in the code.

Interview Questions

- What does it mean for an object to be hashable, and how does that relate to mutability?
- Why can't lists be dictionary keys, but tuples can?
- If you pass a list into a function and the function appends to it, does the caller see the change? What about a tuple?
- What's the difference between `==` and `is` in Python, and when would you use each?
- Explain the mutable default argument bug and how to avoid it.
- Is a tuple containing a list "fully immutable"? Why or why not?
- How would you make a deeply immutable structure in Python?

Similar Problems

- **LeetCode 705 — Design HashSet**: reinforces why hashability and object identity matter for building your own hash-based structure.
- **LeetCode 49 — Group Anagrams**: commonly solved by using a sorted tuple of characters as a dict key — a direct real-world use of tuple hashability.
- **LeetCode 1010 — Pairs of Songs With Total Durations Divisible by 60**: another problem where using tuples (or plain ints) as dict keys is essential because they're hashable.
- **"Design a LRU Cache" (LeetCode 146)**: touches on the same mutable-object-in-place-editing concept, since the cache relies on mutating existing structures rather than rebuilding them.

Key Takeaways

- A variable is a label pointing at an object, not a container holding a value.
- Mutable objects (lists, dicts, sets) can be edited in place — same `id()`, new contents.
- Immutable objects (tuples, strings, ints, frozensets) can never be edited in place — any "change" requires building a new object with a new `id()`.
- Immutability is what makes hashability possible, and hashability is what makes an object usable as a dict key or set member.

- Immutability only locks the container's own slots — a mutable object nested inside an immutable one is still fully mutable.
- Choose a tuple when the shape of your data is fixed and meaningful (coordinates, database rows); choose a list when you expect to grow, shrink, or reorder it.

Watch the Video

For the full walkthrough with live `id()` comparisons and the exact crash in action, watch the video here: {LINK}

About the Series

This lesson is part of *Fun with Learning Technology*, a series built around exactly the kind of gap that trips up developers at every level: concepts you technically "know" but have never seen proven. Each video takes one deceptively simple question, shows you the code that breaks, and builds the intuition back up from first principles instead of just handing you a rule to memorize.

Call To Action

If this cleared up something that's been fuzzy for a while, do two things: hit like on the video so more developers see it, and subscribe on YouTube so you don't miss the next one. For deeper written breakdowns like this one delivered straight to your inbox, subscribe to the Substack. And if you've been bitten by the shallow-immutability gotcha before, drop a comment — that story helps the next person avoid the same bug.

Quick Review

Why does `t=(1,2); t[0]=5` crash but `l=[1,2]; l[0]=5` doesn't?

Lists are mutable (support item assignment); tuples are immutable and raise TypeError on any attempt to change an element.

What does 'mutable' actually mean for an object?

Its internal state can change after creation while keeping the same identity (same id()) — the box stays, contents change.

What proves a list changed in place instead of creating a new object?

id(lst) before and after l.append(x) or l[0]=x stays identical — same memory address, different contents.

Why can't tuples support item assignment at all?

CPython's tuple struct has no mutation methods/slots by design; it's a fixed, read-only array of pointers set once at creation.

What's the payoff of immutability for hashability?

Tuples (with hashable elements) can be dict keys/set members because their hash never changes; mutable lists can't since a changing hash would break lookups.

Gotcha: is a tuple containing a list fully immutable?

No — the tuple's slots are fixed, but a list *inside* it can still be mutated in place; immutability is shallow, not deep.
